Article 1

Title: ResolverFuzz: Automated Discovery of DNS Resolver Vulnerabilities with Query–Response Fuzzing

Authors: Qifan Zhang, Xingyi Bai, Xiaojie Li, Hui Duan, Qing Li, Zhen Li

Published: 33rd USENIX Security Symposium

Year: 2024

Link: https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-qifan

Summary

When a user browses an address online, the website has an IP address, identified through lookups done by a piece of software called a DNS resolver. This resolver also keeps track of what and where it has visited before, making it vulnerable to cybersecurity issues. So, this is where ResolverFuzz can be used to greatly reduce any risks.

Traditional Fuzzers are an already existing tool that test DNS resolvers by throwing in random inputs at the program to check for crashes or other issues encountered. Going beyond a typical fuzzer, ResolverFuzz employs a more diligent and surgical approach since it focuses on meaningful exchanges that could include only a couple of queries and responses to trigger hidden issues rather than a 'brute-forced' array of random packets like a normal fuzzer.

ResolverFuzz has a unique way of functioning which we look into:

1. Short-Sequence Fuzzing

   The professionals working on this tool observed that DNS resolver bugs pop up within only two to three messages, so rather than launching a barrage of hundreds of random packets, ResolverFuzz specifically looks at the ones with interactions. This saves a lot of time and resources, allowing the resolver to be a lot more efficient.

2. Grammar Plus Mutations

   DNS communication must follow a specific protocol and any slight deviation from the protocols could result in bugs. So ResolverFuzz uses a 'Probabilistic Context-Free-Grammar' and encodes the DNS communication format as a grammar which can validate the queries and the responses to make sure it has a wide coverage of the legitimate communication messages. After a message, that follows protocol, is created, the ResolverFuzzer would go and randomly manipulate bytes to make them into variations of the byte which can explore the cases that the grammar misses, like unusual flag combinations or something like a field off by one.

3. Differential Testing with Clustering

   The bugs that pop up in the DNS resolver normally show up as behavioral issues, they typically diverge from expected behavior so we know that there's a bug. So if we compare the implementations of the resolver we can notice differences. ResolverFuzz does this by sending the same fuzzed messages to different resolver softwares (the paper mentions BIND and Unbound). It checks each resolver's cache and the queries and responses stored in them. If it detects any odd behavioral patterns then it flags it as a potential vulnerability. An example of the odd behavior would be like if a resolver accepts a message that other resolvers reject or maybe gets stuck in a loop, then it is flagged. After analyzing any anomalies, ResolverFuzz manages the flagged instances by grouping similar instances together so it can make bugs easy to sort through and organize them to streamline the resolving process.

To catch a wide range of divergent behavior, ResolverFuzz uses something called oracles which are automated detection tools that check for:

- Resources exhaustion, which it can see by monitoring the memory and CPU.
- Crashes and zombie processes.
- Cache poisoning, where it compares the cache of two resolvers to see if it matches.

After running over 700,000 test cases with six resolver programs (BIND, Unbound, Knot, PowerDNS, MaraDNS, Technitium), these professionals found 23 new vulnerabilities using ResolverFuzz. These vulnerabilities included:

- 1 crash bug
- 9 Resource exhaustion problems where the query messages would bloat the memory or get the resolver trapped in loops
- 13 Cache-poisoning cases, where they describe one as 'Out-of-bailiwick' that leaves the system vulnerable and lets attackers inject false records for top-level domains.

We can conclude that ResolverFuzz tailors its approach to detection by appealing to the 'stateful' properties of DNS resolvers and using techniques to generate messages, shift through noise and highlight semantic flaws that the traditional fuzzers could miss. Beyond this, the professionals reached out to vendors to confirm or fix 19 of the bugs and assigned 15 CVEs which clearly shows that there were discrepancies in traditional fuzz methods in resolvers.